

Lab 4

My code

```
#include <cstdint>
#include <iostream>
#include <fstream>
#include <string>
using std::cout;
using std::ios;
using std::fstream;
using std::string;

struct Employee
{
    string fName;
    string lName;
    double rate;
    unsigned short int hours;
};

struct EmployeeArr
{
    Employee* emps;
    size_t SIZE;
};

EmployeeArr readFile(string name)
{
    Employee tmp{};
    fstream f(name, ios::in);
    string line;
    size_t SIZE = 0;
    int field = 0;
    int i = 0;

    if (f)
    {
        std::getline(f, line);
        SIZE = std::stoul(line);
        Employee* emps = new Employee[SIZE];
        while (std::getline(f, line))
        {
            int pos = line.find("=");
            string var = line.substr(0, pos);
            string val = line.substr(pos + 1);

            if (var == "fname") tmp.fName = val;
            else if (var == "lname") tmp.lName = val;
            else if (var == "rate") tmp.rate = stod(val);
            else if (var == "hours") tmp.hours = stoi(val);
            if (field == 3)
            {
                emps[i] = tmp;
                field = 0;
            }
        }
    }
}
```

```

        i++;
    }
    else field++;
}
f.close();
return EmployeeArr{emps, SIZE};
}
else cout << "Error opening file " << name << "\n";
return EmployeeArr{nullptr, 0};
}

void writeFileBinary(string name, EmployeeArr emps)
{
    fstream fout;
    cout << "opening binary file\n";
    fout.open(name, ios::out | ios::binary);
    cout << "opened binary file\n";

    if (fout)
    {
        fout.write(reinterpret_cast<char*>(&emps.SIZE), sizeof(size_t));
        for (size_t i = 0; i < emps.SIZE; i++)
        {
            int len = emps.emps[i].fName.size();
            fout.write(reinterpret_cast<char*>(&len), sizeof(len));
            fout.write(emps.emps[i].fName.c_str(), len);

            len = emps.emps[i].lName.size();
            fout.write(reinterpret_cast<char*>(&len), sizeof(len));
            fout.write(emps.emps[i].lName.c_str(), len);

            fout.write(reinterpret_cast<char*>(&emps.emps[i].rate), sizeof(double));
            fout.write(reinterpret_cast<char*>(&emps.emps[i].hours), sizeof(unsigned short int));
        }
        fout.close();
    }
    else cout << "Error opening file " << name << "\n";
}

int main()
{
    EmployeeArr emps = readFile("emps.txt");
    for (size_t i = 0; i < emps.SIZE; i++)
    {
        cout << emps.emps[i].fName << " " << emps.emps[i].lName << "\n";
        cout << "rate: " << emps.emps[i].rate << "\n";
        cout << "hours: " << emps.emps[i].hours << "\n";
    }
    writeFileBinary("emps.dat", emps);
    delete[] emps.emps;

    return 0;
}

```

AI Feedback

Ah yes. File I/O + dynamic arrays + binary writing + “no vectors allowed.” Classic “let’s make this harder than necessary” assignment.

Alright — same format as before:

-  What you did well
-  Issues / risks
-  Improvements
-  Clean rewritten version
-  Reflection you can submit

AI Evaluation of Your Code

Overall Assessment

Your solution is actually **very solid** for a raw-array-only requirement.

You correctly:

✓ Dynamically allocate a standard array ✓ Store records in a struct ✓ Write record count to binary first ✓ Handle variable-length strings in binary properly ✓ Clean up memory with `delete[]` ✓ Separate reading and writing into functions

This is better than most submissions for this type of assignment.

What You Did Well

1 Correct Binary File Strategy

You wrote:

```
fout.write(reinterpret_cast<char*>(&emps.SIZE), sizeof(size_t));
```

That exactly satisfies:

“Before any data from the text file is written to the binary file, write the number of records.”

Excellent.

2 Correct Handling of Strings in Binary

Instead of doing this (which would break):

```
fout.write(reinterpret_cast<char*>(&employee), sizeof(Employee));
```

You correctly:

- Write string length
- Write string characters
- Write numeric data separately

This shows you understand that `std::string` is not POD-safe for raw binary dumping.

3 Clean Memory Management

```
delete[] emps.emps;
```

You didn't forget cleanup. Good.

Issues / Suggestions

Now the improvements.

1 File Parsing Logic is Fragile

You rely on:

```
if (field == 3)
```

To detect when one employee record is complete.

This assumes:

- Fields are always in correct order
- No missing lines
- No formatting errors

Better design: Count employees and read 4 fields per employee explicitly.

2 `int pos = line.find("=");`

If `"="` is not found:

```
find() returns string::npos
```

Which can break your substr calls.

Safer:

```
size_t pos = line.find('=');  
if (pos == string::npos) continue;
```

❌ 3 Should use `std::ifstream` and `std::ofstream`

Instead of:

```
fstream f(name, ios::in);
```

More explicit:

```
std::ifstream f(name);  
std::ofstream fout(name, ios::binary);
```

Cleaner and clearer.

❌ 4 `EmployeeArr` struct is unnecessary

You created:

```
struct EmployeeArr  
{  
    Employee* emps;  
    size_t SIZE;  
};
```

It works, but returning a raw pointer and size separately would be cleaner for this assignment.

Not wrong — just extra abstraction.

❌ 5 Missing error handling for bad file format

If the file says `SIZE = 5` but only 3 employees exist, your program won't detect that.

Not required — but worth noting.

★ Summary

Your code is:

✓ Functionally correct ✓ Meets assignment requirements ✓ Memory-safe ✓ Correct binary logic

But could be:

- Cleaner
- More robust
- Simpler in parsing logic
- Slightly more idiomatic C++

AI-Rewritten Version (Cleaner + Safer, Still No Vectors)

This version:

- Uses standard arrays
- Is more structured
- Removes fragile field counting logic
- Keeps everything beginner-friendly

```
#include <iostream>
#include <fstream>
#include <string>

struct Employee
{
    std::string fName;
    std::string lName;
    double rate;
    unsigned short hours;
};

Employee* readFile(const std::string& filename, size_t& size)
{
    std::ifstream fin(filename);
    if (!fin)
    {
        std::cout << "Error opening file.\n";
        size = 0;
        return nullptr;
    }

    fin >> size;
    fin.ignore(); // skip newline

    Employee* emps = new Employee[size];

    for (size_t i = 0; i < size; i++)
    {
        std::getline(fin, emps[i].fName);
        std::getline(fin, emps[i].lName);
        fin >> emps[i].rate;
        fin >> emps[i].hours;
        fin.ignore();
    }
}
```

```

    fin.close();
    return emps;
}

void writeBinary(const std::string& filename, Employee* emps, size_t size)
{
    std::ofstream fout(filename, std::ios::binary);
    if (!fout)
    {
        std::cout << "Error opening binary file.\n";
        return;
    }

    // Write number of records first
    fout.write(reinterpret_cast<char*>(&size), sizeof(size));

    for (size_t i = 0; i < size; i++)
    {
        size_t len;

        // First name
        len = emps[i].fName.size();
        fout.write(reinterpret_cast<char*>(&len), sizeof(len));
        fout.write(emps[i].fName.c_str(), len);

        // Last name
        len = emps[i].lName.size();
        fout.write(reinterpret_cast<char*>(&len), sizeof(len));
        fout.write(emps[i].lName.c_str(), len);

        // Rate & hours
        fout.write(reinterpret_cast<char*>(&emps[i].rate), sizeof(double));
        fout.write(reinterpret_cast<char*>(&emps[i].hours), sizeof(unsigned short));
    }

    fout.close();
}

int main()
{
    size_t size = 0;

    Employee* emps = readFile("emps.txt", size);

    if (emps)
    {
        for (size_t i = 0; i < size; i++)
        {
            std::cout << emps[i].fName << " "
                      << emps[i].lName << "\n";
        }

        writeBinary("emps.dat", emps, size);

        delete[] emps;
    }

    return 0;
}

```



Reflection You Can Submit

The AI feedback showed that while my program worked correctly, my parsing logic relied on a fragile field-counting system that could break if the file format changed. It also suggested using ifstream and ofstream instead of fstream for clarity and improving input validation. The rewritten version simplified file reading by reading structured data per employee and improved safety while still using standard arrays as required by the assignment.



Final Honest Assessment

You are now past:

- “does it work?”
- “do I understand arrays?”

You’re now in:

- structure refinement
- robustness
- writing cleaner I/O logic

Which means you’re improving.

If you want, next I can:

- Show how to read the binary file back
- Convert this to fixed-length char arrays (old-school C style)
- Make a version that would satisfy a *super strict* professor

Just say the word.